

Computación Paralela y Distribuida

Ejercicio 30 de Septiembre

Ejercicio 1. Reunir resultados de procesos

En este ejercicio se trabajará con dos formas de reunir información generada por varios procesos:

- `MPI_Gather()`: permite recopilar en un proceso los resultados individuales generados por todos los demás procesos.
- `MPI_Reduce()`: permite combinar esos resultados mediante una operación, por ejemplo suma, máximo o mínimo

Caso 1. Recolección de temperaturas con `MPI_Gather()`

Cada sucursal cuenta con un sensor de temperatura y obtiene una medición local. La Oficina Central necesita recopilar las temperaturas de todas las sucursales para conservar los valores individuales. Analiza y ejecuta el programa `Ejercicio1a.c`, y completa la información solicitada en los siguientes incisos.

- a) **Identificación de los parámetros de `MPI_Gather()`:** especifica qué valor fue asignado a cada parámetro en la llamada, donde se reúnen las temperaturas de todos los procesos en rank 0:

MPI_Gather()		
Parámetro	Valor utilizado en el programa	Explicación: qué representa el valor
<code>sendbuf</code>	<code>&temperatura</code>	Dirección de la variable local de cada proceso (su medición) que será enviada al root.
<code>sendcount</code>	1	Cantidad de elementos que cada proceso envía, una sola temperatura.
<code>sendtype</code>	<code>MPI_FLOAT</code>	Tipo de dato de los elementos enviados, coincide con float temperatura.
<code>recvbuf</code>	temperaturas	Arreglo de 4 float en rank 0 donde se almacenan los valores reunidos. Solo es significativo en el proceso root.
<code>recvcount</code>	1	Cantidad de elementos que el root recibe de cada proceso (no el total).

recvtype	MPI_FLOAT	Tipo de dato de los elementos recibidos, coincide con float temperaturas[].
root	0	Rank del proceso que recibe y reúne los datos.
communicator	MPI_COMM_WORLD	Comunicador que agrupa a todos los procesos participantes; todos deben llamar a MPI_Gather sobre él.

Resultado del programa original (4 procesos): cada proceso imprime su temperatura (24.5, 26.1, 23.8 y 27.0 °C) y la Oficina Central muestra el arreglo reunido en el orden de los ranks 0, 1, 2 y 3

Figura 1. Compilación y ejecución de Ejercicio1a.c (código original) con MPI_Gather()

```

dulce@Dulce:/mnt/c/WINDOWS/system32$ cd "/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida"
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ mpicc Ejercicio1a.c -o Ejercicio1a
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ mpirun -np 4 ./Ejercicio1a
Proceso 1: temperatura registrada = 26.1 C
Proceso 0: temperatura registrada = 24.5 C
Proceso 3: temperatura registrada = 27.0 C
Proceso 2: temperatura registrada = 23.8 C

Oficina Central: temperaturas recibidas
Proceso 0: 24.5 C
Proceso 1: 26.1 C
Proceso 2: 23.8 C
Proceso 3: 27.0 C
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$

```

- b) Modifica el programa: para que cada proceso registre dos mediciones de temperatura y la Oficina Central reúna todos los valores utilizando una sola llamada a MPI_Gather().

Se modificó el programa para que cada proceso registre dos temperaturas y la Oficina Central las reúna con una sola llamada a MPI_Gather(). Cambios realizados:

- temperatura pasó de float a float temperatura[2] y temperaturas de float[4] a float temperaturas[8] (4 procesos × 2 mediciones).
- Cada proceso asigna sus dos valores: P0 = 24.5 y 25.0; P1 = 26.1 y 26.4; P2 = 23.8 y 24.1; P3 = 27.0 y 27.3.
- sendcount y recvcount cambiaron de 1 a 2. Como temperatura ahora es un arreglo, se pasa sin &.
- La impresión del root recorre temperaturas[2*i] y temperaturas[2*i + 1] para mostrar las dos mediciones de cada proceso.

Resultado del programa modificado (4 procesos): la Oficina Central recibió las ocho temperaturas agrupadas por proceso: P0 → 24.5 y 25.0; P1 → 26.1 y 26.4; P2 → 23.8 y 24.1; P3 → 27.0 y 27.3 , tal como se esperaba.

Figura 2. Compilación y ejecución de Ejercicio1a.c modificado (dos mediciones por proceso)

```
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ mpicc modificado/Ejercicio1a.c -o Ejercicio1a_mod
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ mpirun -np 4 ./Ejercicio1a_mod
Proceso 1: temperaturas registradas = 26.1 C, 26.4 C
Proceso 2: temperaturas registradas = 23.8 C, 24.1 C
Proceso 3: temperaturas registradas = 27.0 C, 27.3 C
Proceso 0: temperaturas registradas = 24.5 C, 25.0 C

Oficina Central: temperaturas recibidas
Proceso 0: 24.5 C, 25.0 C
Proceso 1: 26.1 C, 26.4 C
Proceso 2: 23.8 C, 24.1 C
Proceso 3: 27.0 C, 27.3 C
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ _
```

c) Preguntas de análisis:

- i. **¿Por qué MPI_Gather() conserva los resultados individuales de cada proceso?**

Porque MPI_Gather() no combina ni transforma los datos, solo copia el bloque que aporta cada proceso (sendcount elementos) a una zona propia y contigua de recvbuf en el root. Al no aplicarse ninguna operación sobre los valores, cada medición sigue disponible por separado, de modo que la Oficina Central obtiene las temperaturas de las 4 ubicaciones y no un valor resumido.

- ii. **¿Qué relación existe entre la posición de los datos almacenados en recvbuf y el rank de cada proceso?**

Los bloques se ordenan en recvbuf según el rank del proceso emisor, no según el orden en que terminan o imprimen. El bloque del proceso con rank i empieza en la posición $i \times \text{recvcount}$. En el programa original ($\text{recvcount} = 1$) temperaturas[i] corresponde al proceso i (por ejemplo temperaturas[2] = 23.8, del proceso 2). En la versión modificada ($\text{recvcount} = 2$) el proceso i ocupa las posiciones $2i$ y $2i + 1$.

- iii. **Si cada proceso enviara dos valores en lugar de uno, ¿qué cambios serían necesarios en los parámetros de MPI_Gather()?**

- sendcount pasa de 1 a 2: cada proceso envía dos elementos.
- recvcount pasa de 1 a 2: es la cantidad recibida por proceso, no el total (poner 8 sería un error).
- sendbuf debe apuntar a un arreglo de al menos 2 float (temperatura) y recvbuf debe tener espacio para $\text{size} \times \text{recvcount} = 8$ float (temperaturas[8]).
- sendtype, recvtype, root y communicator no cambian (MPI_FLOAT, 0, MPI_COMM_WORLD).

Caso 2. Recolección del consumo eléctrico con MPI_Reduce()

- a) Identifica Identificación de los parámetros de `MPI_reduce()`: especifica qué valor fue asignado a cada parámetro en la llamada a `reduce` en el código compartido, donde se suman los consumos de todos los procesos.

MPI_Gather()		
Parámetro	Valor utilizado en el programa	Explicación: qué representa el valor
<code>sendbuf</code>	<code>&consumo</code>	Dirección de la variable local con el consumo eléctrico de cada proceso (el dato que cada uno aporta a la reducción).
<code>recvbuf</code>	<code>&consumo_total</code>	Variable donde el root guarda el resultado combinado. Solo es significativa en rank 0.
<code>count</code>	1	Número de elementos que cada proceso aporta, el resultado también tiene 1 elemento.
<code>datatype</code>	<code>MPI_INT</code>	Tipo de dato de los valores, coincide con <code>int</code> consumo.
<code>op</code>	<code>MPI_SUM</code>	Operación de reducción aplicada a los valores de todos los procesos, suma (150 + 120 + 180 + 100).
<code>root</code>	0	Rank del proceso que recibe el resultado final, la Oficina Central.
<code>communicator</code>	<code>MPI_COMM_WORLD</code>	Comunicador con todos los procesos que participan en la reducción.

Resultado del programa original (4 procesos): los consumos individuales fueron 150, 120, 180 y 100 kWh, y la Oficina Central obtuvo un consumo total de 550 kWh.

Figura 3. *Compilación y ejecución de Ejercicio1b.c (código original) con `MPI_Reduce()` y `MPI_SUM`*

```

dulce@Dulce:~/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ mpicc original/Ejercicio1b.c -o Ejercicio1b
dulce@Dulce:~/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ mpirun -np 4 ./Ejercicio1b
Proceso 2: consumo = 180 kwh
Proceso 0: consumo = 150 kwh
Proceso 1: consumo = 120 kwh
Proceso 3: consumo = 100 kwh

Consumo total: 550 kwh
dulce@Dulce:~/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$

```

- b) Modifica el programa: para que la Oficina Central obtenga, incluya el consumo total.

Se agregaron dos llamadas adicionales a `MPI_Reduce()` sobre el mismo `sendbuf` (`&consumo`), cada una con su propia variable de resultado y su propia operación:

- consumo_maximo con MPI_MAX: mayor consumo registrado entre los procesos.
- consumo_minimo con MPI_MIN: menor consumo registrado entre los procesos.
- La Oficina Central imprime las tres cantidades con el formato solicitado.

Resultado del programa modificado (4 procesos): la Oficina Central mostró Consumo total: 550 kWh, Consumo maximo: 180 kWh (proceso 2) y Consumo minimo: 100 kWh (proceso 2).

Figura 4. Compilación y ejecución de Ejercicio1b.c modificado (total, máximo y mínimo)

```
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ mpicc modificado/Ejercicio1b.c -o Ejercicio1b_mod
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ mpirun -np 4 ./Ejercicio1b_mod
Proceso 0: consumo = 150 kWh
Proceso 2: consumo = 180 kWh
Proceso 3: consumo = 100 kWh
Proceso 1: consumo = 120 kWh
Consumo total: 550 kWh
Consumo maximo: 180 kWh
Consumo minimo: 100 kWh
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$
```

c) Preguntas de análisis

i. ¿Qué función cumple el parámetro op dentro de MPI_Reduce()?

op indica la operación con la que se combinan los valores aportados por todos los procesos para producir un único resultado en el root. Sin él, MPI_Reduce() no sabría cómo reducir los datos. Es lo que diferencia una reducción de una simple recolección, MPI aplica op elemento a elemento sobre los valores de todos los procesos y entrega el resultado combinado en recvbuf del rank root.

ii. ¿Qué diferencia existe entre utilizar MPI_SUM, MPI_MAX y MPI_MIN?

Las tres llamadas usan los mismos datos y la misma firma; solo cambia la operación y, con ello, el significado del resultado:

Operación	Qué calcula	Resultado en este ejercicio
MPI_SUM	Suma de los valores de todos los procesos.	$150 + 120 + 180 + 100 = 550$ kWh
MPI_MAX	El mayor de los valores.	180 kWh (proceso 2)
MPI_MIN	El menor de los valores.	100 kWh (proceso 2)

iii. ¿Cuál es la diferencia conceptual entre MPI_Gather() y MPI_Reduce()?

MPI_Gather() recolecta el root recibe todos los valores individuales, ordenados por rank, en un buffer de size $\times$ recvcoun elementos y puede consultar el dato de cada proceso. MPI_Reduce() combina:

aplica una operación (op) a los valores de todos los procesos y el root recibe un solo resultado de count elementos, sin posibilidad de recuperar los valores individuales a partir de él. Se usa MPI_Gather() cuando importa cada dato (temperaturas por sucursal) y MPI_Reduce() cuando solo interesa un valor agregado (consumo total, máximo o mínimo). Además, MPI_Gather() no recibe op.

LINK DEL REPOSITORIO:

La solución implementada se encuentra en el siguiente [repositorio](#) (clic en la imagen):



LINK DEL VIDEO: <https://youtu.be/6XZmSMrjfog>

